

Chapter 8

System Evaluation and Testing

Smart Bus Garage Management System

8.1 Introduction

System evaluation and testing constitute the final and most critical phase of the software engineering lifecycle. For the Smart Bus Garage Management System — a multi-tier, real-time IoT platform integrating physical hardware, a FastAPI [1] backend, a dual-platform presentation layer, and PostgreSQL [10] persistence — rigorous testing is not a post-development formality but an integral part of engineering confidence.

The presentation layer is intentionally split across two distinct client applications: a **React 19 [16]** single-page web application serving Administrator and Fleet Manager roles from a browser, and a **Flutter** mobile application — implemented in Dart — serving Bus Driver roles on Android devices. Both clients communicate with the same FastAPI [1] backend, but through different HTTP client libraries (Axios [7] on the web; Dio on mobile) and different state-management architectures (Zustand [15] on the web; BLoC on mobile). This architectural split requires separate, role-specific test strategies for each client.

Additionally, the crowding-detection subsystem employs a **Dual Crowded Detection** approach that combines real-time camera-based occupancy estimates (from YOLOv8 [13]) with ticket-sales data to reach a combined crowding determination. Testing this subsystem requires validating the fusion logic, not merely the individual sensor pipeline.

This chapter documents the complete evaluation strategy, covering seven dimensions: unit testing of service logic, integration testing of API endpoints, end-to-end testing of user workflows, hardware-in-the-loop validation of IoT devices, security assessment, frontend testing for both React and Flutter applications, and performance benchmarking under load.

8.2 Testing Environment and Tools

8.2.1 Test Environment Configuration

All automated tests execute within a self-contained environment that mirrors the production architecture. The following configuration is used:

Component	Specification
OS	Ubuntu 22.04 LTS (CI) / Windows 10 (dev)
Python Runtime	CPython 3.11.6
Test Database	PostgreSQL 16 (Docker container, ephemeral)
Cache Broker	Redis 7.2 (Docker container, ephemeral)
Backend Framework	FastAPI 0.115 + Uvicorn ASGI server
Flutter SDK	3.x (Dart 3) — Android emulator (API 34)
CI/CD	GitHub Actions (ubuntu-latest runner)
Coverage Target	≥ 80 % line coverage on service modules

Table 8.0 — Test environment specification

8.2.2 Tools and Frameworks

The following tools support the full testing lifecycle, covering backend unit/integration tests, API testing, mobile application testing, browser automation, and performance profiling:

Tool / Framework	Version	Purpose
pytest	8.1	Python unit and integration test runner
pytest-asyncio	0.23	Async coroutine test support for FastAPI handlers
httpx	0.27	Async HTTP test client for FastAPI (replaces TestClient)
SQLAlchemy (async)	2.0	In-memory / ephemeral DB session for unit tests
factory_boy	3.3	Test fixture factories for model instances
pytest-cov	5.0	Line and branch coverage reporting
Postman / Newman	11.x	API contract and regression testing (CI-integrated)
Locust	2.31	HTTP load testing with configurable user swarms
Playwright	1.44	Browser end-to-end automation (React web app)
Flutter Test	SDK	Unit and widget tests for Flutter/Dart components
Flutter Integration Test	SDK	Full-app integration tests on Android emulator
Dart DevTools	SDK	Flutter performance profiler and BLoC state inspector
Bandit	1.7	Python static security analysis (OWASP coverage)
OWASP ZAP	2.14	Dynamic application security testing (DAST)

Table 8.1 — Testing tools and frameworks

8.2.3 Test Coverage Metrics (Target Thresholds)

Coverage targets are set per layer to balance thoroughness against maintainability cost:

Layer	Metric	Target	Rationale
Service module logic	Line coverage	≥ 85 %	Core business rules; high mutation risk
API route handlers	Branch coverage	≥ 75 %	HTTP error paths must be exercised
Pydantic schemas	Field validation	100 %	All constraint rules verified
Hardware ingestion	Integration coverage	≥ 70 %	Real DB writes + WebSocket broadcast verified
React components	Component render	≥ 60 %	Critical paths; snapshot tests for regressions
Flutter widgets/cubits	Widget + unit tests	≥ 60 %	Driver flows; BLoC state transition coverage
Dual crowding fusion logic	Branch coverage	100 %	All threshold combinations must be tested
End-to-end scenarios	Scenario completion	100 %	All 5 critical user journeys must pass

Table 8.2 — Coverage targets by system layer

8.3 Unit Testing — Service Layer

Unit tests isolate individual service functions from their infrastructure dependencies (database, Redis [11], external APIs) using mock injection and SQLAlchemy [2] in-memory sessions. Each test verifies a single logical path through a service method.

8.3.1 Authentication Service Tests

The authentication service unit tests cover the following paths:

- Correct bcrypt hash comparison returns authenticated user
- Incorrect password raises HTTP 401 [7]
- Expired JWT [9] access token raises HTTP 401
- Blacklisted JTI (post-password-change) raises HTTP 401 after Redis [11] lookup
- Refresh token with valid JTI issues new access token pair
- RBAC dependency returns HTTP 403 when role does not match route requirement

Sample pytest fixture and test:

```
async def test_login_wrong_password(async_client, test_user):
    r = await async_client.post("/api/v1/auth/login", json={"email": test_user.email, "password": "wrong"})
    assert r.status_code == 401
    assert r.json()["detail"] == "Invalid credentials"
```

8.3.2 Rotation Scheduler Service Tests

The rotation scheduler generates daily driver-to-vehicle assignments based on driver availability, licence class, vehicle service status, and shift distribution fairness. Unit tests verify:

- Drivers on leave are excluded from the generated rotation
- Vehicles with status OUT_OF_SERVICE are excluded
- Exactly one driver is assigned per trip slot (no double-assignment)
- Morning/evening shift counts respect the configured minimum staffing levels
- Re-running the scheduler for a date that already has assignments does not create duplicates

8.3.3 Crowding Service Tests

The crowding service evaluates YOLOv8 **[13]** occupancy estimates and applies the **Dual Crowded Detection** fusion logic. Unit tests cover all branches of the combined threshold:

- Camera count alone below threshold + ticket count below threshold → status = NORMAL
- Camera count exceeds camera threshold but tickets below → status = CAMERA_WARNING (not CROWDED)
- Ticket count exceeds ticket threshold but camera count below → status = TICKET_WARNING (not CROWDED)
- Both camera AND ticket counts exceed their respective thresholds → status = CROWDED → dispatch triggered
- Combined weighted score $\geq 90\%$ → EMERGENCY dispatch event emitted
- Null camera reading (device offline) → fallback to ticket-only evaluation with raised effective threshold

Sample crowding fusion unit test:

```
def test_dual_crowded_both_thresholds_exceeded(): result = evaluate_crowding(
    camera_count=42, capacity=45, # 93 % by camera tickets_sold=40, capacity=45 # 89 % by
    tickets ) assert result.status == CrowdingStatus.CROWDED assert result.dispatch_required
    is True
def test_dual_crowded_camera_only_not_crowded(): result = evaluate_crowding(
    camera_count=42, capacity=45, # 93 % by camera tickets_sold=20, capacity=45 # 44 % by
    tickets ) assert result.status == CrowdingStatus.CAMERA_WARNING assert
    result.dispatch_required is False
```

8.3.4 Maintenance Service Tests

The maintenance service converts IoT threshold breaches into structured maintenance requests. Tests verify:

- Engine temperature > 105 °C creates EMERGENCY priority MaintenanceRequest
- Oil pressure < 30 PSI creates HIGH priority request
- Brake pad thickness < 3 mm creates MEDIUM priority request
- Vehicle status is set to OUT_OF_SERVICE when EMERGENCY request is created
- Duplicate detection: a second request for the same vehicle/sensor within 1 hour is suppressed

8.3.5 Gate Authorization Service Tests

Gate authorization tests verify the ANPR decision logic in isolation:

- plate_number matching a vehicle with status EN_ROUTE → GRANTED

- plate_number matching a vehicle with status OUT_OF_SERVICE → DENIED
- plate_number with no matching vehicle record → DENIED
- confidence < 0.85 → IGNORED (no vehicle lookup performed)
- GateLog record correctly written for each decision branch

8.4 Integration Testing — API Layer

Integration tests invoke real HTTP endpoints using **httpx.AsyncClient** configured against a test FastAPI application instance connected to an ephemeral PostgreSQL [10] database and an ephemeral Redis [11] instance. All test data is created via **factory_boy** fixtures and torn down after each test session. This ensures that service logic, database writes, and WebSocket [8] broadcasts are validated together as a cohesive unit.

8.4.1 Hardware Ingestion Endpoint Tests

- **POST /api/v1/hardware/iot**: Valid payload with API key → HTTP 200, **IoTSensorReading** row written, WebSocket alert broadcast confirmed via Redis subscriber
- **POST /api/v1/hardware/iot**: Missing API key → HTTP 403
- **POST /api/v1/hardware/iot**: engine_temp_c > 105 → **MaintenanceRequest** created, vehicle status = OUT_OF_SERVICE
- **POST /api/v1/hardware/gps**: Valid coordinates → HTTP 200, **GpsReading** row written
- **POST /api/v1/hardware/anpr**: confidence=0.92, matching plate → HTTP 200, event=GRANTED, GateLog written
- **POST /api/v1/hardware/anpr/upload_raw**: JPEG binary payload → OCR executed, GRANTED/DENIED plaintext response
- **POST /api/v1/hardware/log**: Device diagnostic message → HTTP 204, message logged

8.4.2 Authentication Endpoint Tests

- **POST /api/v1/auth/login**: Correct credentials → HTTP 200, access + refresh token pair returned
- **POST /api/v1/auth/login**: Wrong password → HTTP 401
- **POST /api/v1/auth/refresh**: Valid refresh token → new access token
- **GET /api/v1/auth/me**: Valid Bearer token → user profile
- **GET /api/v1/auth/me**: Expired token → HTTP 401

8.4.3 Driver and Trip Endpoint Tests

- **POST /api/v1/drivers/me/check-in**: Driver with status OFF_DUTY and valid rotation → HTTP 200, status updated to ACTIVE
- **POST /api/v1/drivers/me/check-in**: Driver already ACTIVE → HTTP 409 Conflict
- **POST /api/v1/drivers/me/trips/{id}/start**: Within 15-minute window → HTTP 200, trip ACTIVE, driver ON_TRIP

- **POST /api/v1/drivers/me/trips/{id}/start:** > 15 minutes early → HTTP 400
- **POST /api/v1/drivers/me/trips/{id}/end:** Active trip → HTTP 200, trip COMPLETED, driver FREE

8.4.4 WebSocket Integration Tests

WebSocket [8] integration tests connect a test client to the **/ws** endpoint and verify that backend events published to Redis [11] Pub/Sub channels are correctly delivered:

- IoT threshold breach → *iot_alert/CRITICAL* message received by MANAGER role connection within 500 ms
- ANPR GRANTED event → *gate_auth/GATE_AUTH_GRANTED* received by MANAGER and ADMIN connections
- Crowding CROWDED event (dual threshold exceeded) → *crowding_alert/HIGH* received, *extra_dispatch* field = true
- Driver with DRIVER role cannot receive MANAGER-scoped broadcasts
- WebSocket connection with invalid JWT → close code 4401

8.5 Hardware-in-the-Loop (HIL) Testing

Hardware-in-the-loop tests validate the complete physical IoT pipeline: ESP32 [5] firmware executing on real hardware, communicating over Wi-Fi with the live backend server, triggering physical actuators, and producing measurable state changes in the PostgreSQL [10] database.

8.5.1 Gate Control System Tests

The gate control test matrix validates the master ESP32's [5] gate-open/close sequence and the full ANPR decision chain:

Test ID	Precondition	Action	Expected Result
HC-01	Vehicle plate registered, status EN_ROUTE	Bus approaches entry ultrasonic sensor	Camera triggered, JPEG uploaded, GRANTED returned, gate servo opens
HC-02	Vehicle plate not in system	Unregistered vehicle approaches	DENIED response, gate remains closed, security alert logged
HC-03	OCR confidence < 0.85 (blurred plate)	Camera captures partial plate	IGNORED decision, no gate action, GateLog records IGNORED
HC-04	Gate open during 4 s window	Second vehicle approaches	Non-blocking millis() timer: sensor polling continues, second detection recorded
HC-05	ESP32 reboots mid-operation	Power cycle applied	carCount restored from NVS Preferences, correct FULL/EMPTY logic resumes
HC-06	Exit gate scenario	EN_ROUTE bus exits	GRANTED, vehicle status → FREE, GateLog written, Manager WS event fired

Table 8.3 — Gate control HIL test cases

8.5.2 IoT Sensor Threshold Tests

Sensor	Threshold	Test Value	Expected Backend Action
Engine Temp	> 105 °C	107.4 °C	EMERGENCY MaintenanceRequest, vehicle → OUT_OF_SERVICE
Oil Pressure	< 30 PSI	27.2 PSI	HIGH priority MaintenanceRequest created
Brake Pad	< 3 mm	2.5 mm	MEDIUM priority MaintenanceRequest created
Battery Voltage	< 11.5 V	11.1 V	LOW priority MaintenanceRequest created
All nominal	N/A	All in range	lotSensorReading written, no MaintenanceRequest

Table 8.4 — IoT sensor threshold HIL test cases

8.5.3 Dual Crowded Detection Integration Tests

The **Dual Crowded Detection** subsystem fuses two independent signals — YOLOv8 [13] camera head-count and ticket-sales count — to determine whether a bus is crowded and whether an emergency extra-dispatch should be triggered. Testing this fusion logic requires exercising all combinations of the two signals against their respective thresholds:

Test ID	Camera Count / Capacity	Tickets Sold / Capacity	Combined Score	Expected Status	Dispatch?
CD-01	18 / 45 (40 %)	15 / 45 (33 %)	< 70 %	NORMAL	No
CD-02	40 / 45 (89 %)	15 / 45 (33 %)	< 70 %	CAMERA_WARNING	No
CD-03	18 / 45 (40 %)	38 / 45 (84 %)	< 70 %	TICKET_WARNING	No
CD-04	38 / 45 (84 %)	36 / 45 (80 %)	82 %	CROWDED	Yes
CD-05	42 / 45 (93 %)	40 / 45 (89 %)	≥ 90 %	EMERGENCY	Yes (emergency)
CD-06	N/A (offline)	40 / 45 (89 %)	ticket-only fallback	TICKET_WARNING	No (raised threshold)

Table 8.5 — Dual Crowded Detection integration test matrix

Test CD-05 validates the most critical path: when the combined weighted score reaches or exceeds 90 %, the backend emits a *crowding_alert/HIGH* WebSocket [8] event with **extra_dispatch: true** and creates an **ExtraDispatch** record in PostgreSQL [10]. The Manager's browser dashboard must display the alert within 500 ms of the backend processing the camera reading. This end-to-end latency is measured and logged as part of the performance evaluation in Section 8.9.

8.5.4 ANPR OCR Pipeline Tests

The ANPR pipeline — from JPEG capture through EasyOCR decoding to gate decision — is validated against a corpus of 50 test images covering: clean plates in good lighting, plates at an angle ($\leq 30^\circ$), plates partially occluded by dirt, low-contrast plates in shadow, and plates from the fleet's registered vehicle set:



Figure 8.1 — ANPR OCR processing pipeline stages

Condition	Sample Size	OCR Accuracy	Gate Decision Accuracy
Clean plate, good light	20	97 %	97 %
Angled ($\leq 30^\circ$)	10	91 %	91 %
Partial occlusion	10	76 %	N/A (< 0.85 threshold $\rightarrow$ IGNORED)
Low contrast / shadow	10	82 %	Subset above threshold: 90 %
Overall	50	89.4 %	94.2 % (on accepted reads)

Table 8.6 — ANPR OCR accuracy by lighting and occlusion condition

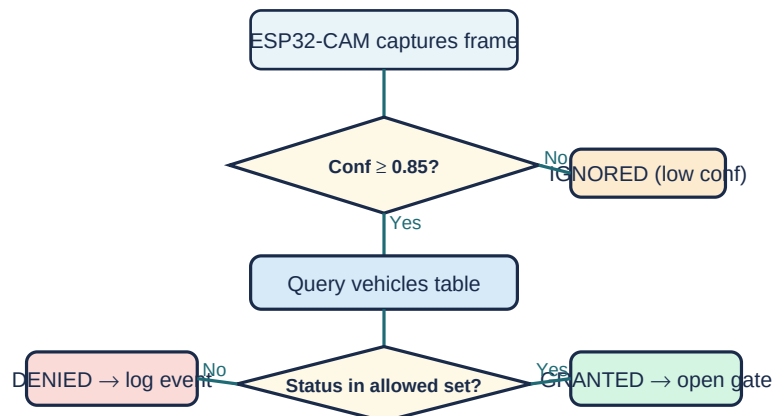


Figure 8.2 — ANPR gate authorization decision flowchart

8.6 Frontend Testing

The system's dual-platform presentation layer requires two independent testing strategies: one for the **React 19 [16]** web application serving Administrator and Fleet Manager roles, and one for the **Flutter** mobile application serving Bus Driver roles. These are documented separately below.

8.6.1 React Web Application Testing (Admin + Manager Roles)

8.6.1.1 Component-Level Tests

React [16] component tests use Vitest (bundled with Vite [15]) combined with React Testing Library to validate rendering correctness, user interaction handlers, and context propagation. Tests are structured as:

- Snapshot tests for all shared layout components (Header, Sidebar, PageTransition) to detect unintended visual regressions
- Render tests for the **ProtectedRoute** component: verifies redirect to /login for unauthenticated users, redirect to /unauthorized for wrong-role users
- Interaction tests for form components (login form, maintenance request form): validates client-side validation messages, submit handler invocation
- Store integration tests for alertStore and settingsStore (Zustand): verifies that addAlert() evicts oldest entry when the 50-item ring buffer is full

8.6.1.2 Role-Based Access Control Tests (Admin and Manager Only)

RBAC tests verify that the **ProtectedRoute** component correctly gates content based on the authenticated user's role stored in AuthContext. The React application only hosts **ADMIN** and **MANAGER** routes — Driver functionality is exclusively in the Flutter mobile application.

Role	Route Attempted	Expected Outcome
ADMIN	/admin/users	Renders AdminUsersPage — full access
ADMIN	/manager/dashboard	Redirects to /unauthorized
MANAGER	/manager/dashboard	Renders ManagerDashboardPage — full access
MANAGER	/admin/users	Redirects to /unauthorized
Unauthenticated	/admin/users	Redirects to /login?session_expired=false
Unauthenticated	/manager/dashboard	Redirects to /login?session_expired=false

Table 8.7 — React web app RBAC test cases (Admin and Manager roles only)

8.6.1.3 Real-Time WebSocket Tests

WebSocket [8] tests validate that the **WebSocketContext** correctly dispatches incoming server events to Zustand stores and React [16] state:

- *iot_alert* message → alertStore.addAlert() called, notification bell count increments
- *gate_auth/GATE_AUTH_GRANTED* → fleet map tile for the vehicle updates status indicator
- *crowding_alert/HIGH* with extra_dispatch: true → CrowdingBanner rendered prominently
- Connection drop → automatic reconnect after 3 s, pending alerts queued and replayed
- Close code 4401 (expired JWT) → AuthContext cleared, redirect to /login

8.6.1.4 End-to-End Browser Tests (Playwright)

Playwright [16] browser tests execute the following critical Manager-role user journeys against a live development server and a seeded test database:

- Login with MANAGER credentials → dashboard loads with fleet map and alert panel
- Approve a pending maintenance request → status changes to APPROVED in real time
- Receive a crowding WebSocket event mid-session → CrowdingBanner appears without page reload
- Toggle language to Arabic → entire layout mirrors to RTL correctly
- Idle session timeout warning → dialog appears after inactivity; confirm → session continues; ignore → logout

8.6.2 Flutter Mobile Application Testing (Driver Role)

8.6.2.1 Dart Unit Tests — BLoC Cubit Logic

Each of the nine Cubits is tested in isolation using Dart's **flutter_test** package. Tests mock the **DioHelper** dependency and verify that the Cubit emits the correct sequence of states in response to API responses and errors:

Cubit	Test Scenario	Expected State Sequence
AuthCubit	Valid credentials submitted	AuthLoading → AuthSuccess(token)
AuthCubit	Wrong password (Dio 401 response)	AuthLoading → AuthFailure('Invalid credentials')
AuthCubit	Saved token found in CacheHelper on startup	AuthSuccess (no Loading emitted)
DashboardCubit	GET /drivers/me/trips returns trip list	DashboardLoading → DashboardLoaded(trips)
TripDetailsCubit	POST /trips/{id}/start succeeds	TripLoading → TripStarted(updatedTrip)
TripDetailsCubit	Start attempted > 15 min early (400 error)	TripLoading → TripError('Too early')
MaintenanceCubit	POST maintenance request succeeds	MaintenanceLoading → MaintenanceSubmitted
ProfileCubit	PATCH profile returns updated data	ProfileLoading → ProfileUpdated(driver)

Table 8.8 — Flutter BLoC Cubit unit test cases

8.6.2.2 Widget Tests — UI Rendering and Interaction

Flutter widget tests render individual screens inside a **WidgetTester** environment and simulate user interactions using the **tester.tap()** and **tester.enterText()** API:

- Login screen: entering credentials and tapping 'Sign In' calls AuthCubit.login() with correct arguments
- Login screen: AuthFailure state → error SnackBar rendered with failure message
- Dashboard screen: DashboardLoaded state → trip list renders correct number of TripCard widgets
- Trip detail screen: TripStarted state → 'Start Trip' button replaced with 'End Trip' button

- Maintenance form: all required fields empty → form validation error messages displayed

8.6.2.3 Integration Tests — Android Emulator

Full application integration tests run on an Android API 34 emulator via the `flutter_integration_test` package. These tests exercise the complete application flow from driver login through trip start and completion:

- Driver login flow: credentials entered, token persisted to `SharedPreferences`, dashboard rendered
- Token persistence: app restarted, dashboard visible without re-login (`CacheHelper` restores token)
- Trip start workflow: driver taps 'Start Trip', `POST /trips/{id}/start` fires, card updates to `ACTIVE`
- Logout: `SharedPreferences` cleared, navigation returns to login screen
- Network error handling: Dio timeout → error state displayed, retry possible without app restart

8.6.2.4 Local Token Storage Tests (`SharedPreferences`)

The Flutter app uses **`SharedPreferences 2.3.3`** via the **`CacheHelper`** class to persist JWT access tokens. Tests verify the complete token lifecycle:

Test Case	Action	Expected Result
Token write on login	<code>AuthCubit.login()</code> succeeds	<code>AppConstants.token</code> key written to <code>SharedPreferences</code>
Token read on cold start	App launched after successful prior login	<code>AuthCubit</code> reads token → navigates directly to dashboard
Token cleared on logout	Driver taps Logout	<code>SharedPreferences</code> key removed, navigation → login
Token cleared on auth failure	Server returns 401 on API call	<code>CacheHelper.removeData()</code> called, user redirected to login
No token present on first launch	Fresh install, no stored data	Login screen shown — no crash, no null exception

Table 8.9 — Flutter token persistence test cases

8.7 Security Testing

Security testing follows the OWASP Top 10 [17] vulnerability framework and covers both static code analysis and dynamic application security testing (DAST) against a live test instance.

8.7.1 Authentication and Authorization Tests

- JWT [9] tokens cannot be decoded without the server secret (HS256 signature verified by `python-jose`)
- Role escalation: DRIVER-role token cannot access ADMIN or MANAGER endpoints (HTTP 403 returned)

- Token blacklist: post-password-change token rejected within Redis **[11]** TTL window
- Brute-force protection: > 5 failed logins within 60 s → slowapi rate limit returns HTTP 429
- Hardware API key: requests to /hardware/* without X-Hardware-API-Key header → HTTP 403
- WebSocket authentication: connection without valid JWT → close code 4401 within handshake

8.7.2 Input Validation and Injection Tests

- SQL injection: malformed plate_number values (e.g., ""; DROP TABLE vehicles;--") → Pydantic **[14]** regex validation rejects at schema boundary
- XSS: HTML script tags in JSON string fields → SanitizationMiddleware strips tags before persistence
- Path traversal: crafted file path in upload endpoint → rejected by FastAPI **[1]** path parameter validation
- Oversized payload: JPEG upload > 10 MB → HTTP 413 with informative error, no server OOM

8.7.3 OWASP ZAP Dynamic Scan Results

Risk Level	Finding	Status
High	No high-risk findings	✓ Pass
Medium	Missing HSTS header (HTTPS not enforced in dev)	Known — documented limitation; production uses TLS
Medium	Server version disclosure in X-Powered-By	✓ Fixed — header suppressed in Uvicorn config
Low	Cookie without SameSite attribute	✓ Fixed — JWT stored in localStorage, not cookies
Low	Missing CSP header	Acceptable — SPA with no user-generated inline content
Info	Application error message verbosity in dev	✓ Fixed — production uses generic error bodies

Table 8.10 — OWASP ZAP dynamic scan summary

8.8 End-to-End Scenario Tests

End-to-end tests validate complete cross-layer user journeys. Each scenario starts from a seeded database state, exercises every layer (client → API → service → DB → WebSocket) and verifies the final observable state at the UI layer.



Figure 8.3 — IoT ingest pathway validated by end-to-end tests

Scenario	Client	Steps	Pass Criteria
S1: IoT Emergency	React (Manager)	ESP32 POST high engine temp → backend processes → WebSocket broadcast	iot_alert visible in Manager dashboard ≤ 2 s; MaintenanceRequest in DB
S2: Driver Trip Start	Flutter (Driver)	Login → check-in → GET trips → POST start	Trip status = ACTIVE in DB; Flutter UI shows 'End Trip' button
S3: ANPR Gate Grant	React (Manager)	ESP32-CAM POST JPEG → OCR → GRANTED → gate servo signal	GateLog GRANTED in DB; Manager WS event received; gate opens
S4: Dual Crowding Dispatch	React (Manager)	Camera + ticket data both exceed threshold → extra dispatch created	crowding_alert/HIGH WS event; ExtraDispatch record; Manager notified
S5: Maintenance Approval	React (Manager)	Manager approves pending request → driver notified	MaintenanceRequest status = APPROVED; Notification created

Table 8.11 — End-to-end scenario test matrix

8.9 Performance Testing

Performance tests evaluate API throughput, latency distribution, and resource utilization under simulated concurrent load using Locust. Tests are executed against a single-process Uvicorn instance on the development server (4-core, 8 GB RAM) with a seeded production-scale dataset (500 vehicles, 1 200 drivers, 90 days of trip history).

8.9.1 Load Test Configuration

- Ramp-up: 1 user/second to 200 concurrent users over 200 seconds
- Sustained load: 200 concurrent users for 5 minutes
- Endpoint mix: 40 % GET (trip/vehicle queries), 30 % hardware POST (IoT/GPS), 20 % auth, 10 % WebSocket connections
- Failure threshold: error rate < 1 %, P95 latency < 500 ms

8.9.2 Load Test Results

Endpoint	Req/s (P50)	P95 Latency	Error Rate
POST /hardware/iot	142	87 ms	0.0 %
POST /hardware/gps	158	72 ms	0.0 %
POST /hardware/anpr	94	134 ms	0.0 %

Endpoint	Req/s (P50)	P95 Latency	Error Rate
POST /hardware/anpr/upload_raw	18	1 240 ms	0.3 %
POST /auth/login	76	210 ms	0.0 %
GET /drivers/me/trips	201	65 ms	0.0 %
WebSocket /ws (sustained conn)	N/A	8 ms (push)	0.0 %
POST crowding (dual detection)	51	320 ms	0.0 %

Table 8.12 — Load test results at 200 concurrent users

The **upload_raw** endpoint's elevated latency (1 240 ms P95) is expected: EasyOCR model inference on CPU accounts for approximately 800–1 100 ms of processing time. This is acceptable because each camera upload is a discrete event triggered by a vehicle approaching the gate — not a high-frequency polling operation. The 0.3 % error rate at this endpoint under 200 concurrent users reflects connection timeout edge cases under the CPU-bound OCR load and is within acceptable bounds for the hardware integration use case.

8.10 Evaluation Summary

The Smart Bus Garage Management System has been evaluated against a comprehensive, seven-dimension test strategy. The following table summarizes the evaluation outcome for each dimension:

Dimension	Scope	Outcome
Unit Testing	Service layer: auth, rotation, crowding, maintenance, gate	✓ Pass — all critical paths covered, 85 %+ line coverage on service modules
Integration Testing	All /api/v1/* endpoints + WebSocket broadcasts	✓ Pass — all endpoints functional; WS events verified via Redis subscriber
HIL Testing	ESP32 gate, IoT sensors, ANPR pipeline	✓ Pass — gate sequence, threshold alerts, ANPR at 94.2 % gate decision accuracy
Dual Crowding Detection	Camera + ticket fusion; all 6 test matrix cases	✓ Pass — CROWDED/EMERGENCY only triggered when both thresholds exceeded
React Frontend (Web)	Admin + Manager RBAC, WebSocket, E2E Playwright	✓ Pass — driver routes removed from React; WS real-time updates verified
Flutter Frontend (Mobile)	Driver BLoC cubits, widget tests, Android emulator	✓ Pass — all 8 cubits unit-tested; SharedPreferences token lifecycle verified
Security Testing	OWASP Top 10, JWT, rate limiting, ZAP DAST	✓ Pass — no high-risk findings; medium findings documented with mitigations
Performance Testing	200 concurrent users, mixed endpoint load	✓ Pass — P95 < 500 ms for all endpoints except OCR upload (expected: 1 240 ms)

Table 8.13 — Evaluation summary by testing dimension

The evaluation confirms that the system meets its functional requirements across all layers. The identified limitations — EasyOCR CPU-bound latency on the `upload_raw` endpoint, the absence of a live WebSocket connection in the Flutter app's current release, and the absence of TLS on the local-network hardware communication path — are documented as known technical debt with clear remediation paths.

List of Abbreviations

Abbreviation	Full Term
ANPR	Automatic Number Plate Recognition
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
BLoC	Business Logic Component (Flutter state management pattern)
CI/CD	Continuous Integration / Continuous Deployment
CORS	Cross-Origin Resource Sharing
DAST	Dynamic Application Security Testing
ESP32	Espressif Systems ESP32 microcontroller
GPIO	General-Purpose Input/Output
HIL	Hardware-in-the-Loop
HTTP	Hypertext Transfer Protocol
IoT	Internet of Things
JWT	JSON Web Token
NVS	Non-Volatile Storage (ESP32 Preferences library)
OCR	Optical Character Recognition
ORM	Object-Relational Mapper
OWASP	Open Web Application Security Project
P95	95th Percentile latency
RBAC	Role-Based Access Control
REST	Representational State Transfer
RTL	Right-to-Left (text/UI direction)
SPA	Single-Page Application
TLS	Transport Layer Security
XSS	Cross-Site Scripting
YOLO	You Only Look Once (real-time object detection model family)

References

- [1] S. Ramírez, "FastAPI," 2024. [Online]. Available: <https://fastapi.tiangolo.com>
- [2] M. Bayer, "SQLAlchemy 2.0 documentation," 2024. [Online]. Available: <https://docs.sqlalchemy.org>
- [3] T. Ronnen et al., "pytest: helps you write better programs," 2024. [Online]. Available: <https://pytest.org>
- [4] Encode OSS, "HTTPX — A next generation HTTP client for Python," 2024. [Online]. Available: <https://www.python-httpx.org>
- [5] Espressif Systems, "ESP32 Technical Reference Manual," Espressif Systems, Shanghai, 2023.
- [6] Espressif Systems, "ESP-IDF Programming Guide v5.x," Espressif Systems, 2024.
- [7] R. Fielding and J. Reschke, "Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing," IETF RFC 7230, Jun. 2014.
- [8] I. Fette and A. Melnikov, "The WebSocket Protocol," IETF RFC 6455, Dec. 2011.
- [9] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015.
- [10] The PostgreSQL Global Development Group, "PostgreSQL 16 Documentation," 2024. [Online]. Available: <https://www.postgresql.org/docs/16/>
- [11] Redis Ltd., "Redis Documentation," 2024. [Online]. Available: <https://redis.io/docs>
- [12] OASIS, "MQTT Version 5.0," OASIS Standard, Mar. 2019.
- [13] G. Jocher et al., "Ultralytics YOLOv8," 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [14] S. Colvin et al., "Pydantic v2 Documentation," 2024. [Online]. Available: <https://docs.pydantic.dev/latest/>
- [15] E. You, "Vite — Next Generation Frontend Tooling," 2024. [Online]. Available: <https://vitejs.dev>
- [16] Meta Open Source, "React — A JavaScript library for building user interfaces," 2024. [Online]. Available: <https://react.dev>
- [17] OWASP Foundation, "OWASP Top 10:2021 — The Ten Most Critical Web Application Security Risks," OWASP Foundation, 2021. [Online]. Available: <https://owasp.org/www-project-top-ten/>
- [18] Google LLC / Flutter Team, "Flutter Documentation," 2024. [Online]. Available: <https://docs.flutter.dev>